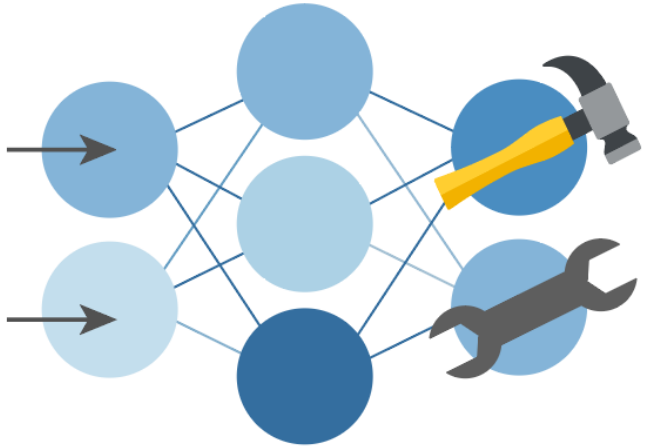
Python + Agents



🛠️ Feb 24: Building your first agent in Python
🧠 Feb 25: Adding context and memory to agents
📊 Feb 26: Monitoring and evaluating agents
🔀 Mar 3: Building your first AI-driven workflows
🕸️ Mar 4: Orchestrating advanced multi-agent workflows
🙋🏽‍♀️ Mar 5: Adding a human-in-the-loop to workflows

🔗 **Register at aka.ms/PythonAgents/series**

Microsoft

Python + Agents

🧠 Adding context and memory to agents

aka.ms/pythonagents/slides/contextmemory

Pamela Fox

Python Cloud Advocate

www.pamelafox.org

Today we'll cover different kinds of context...

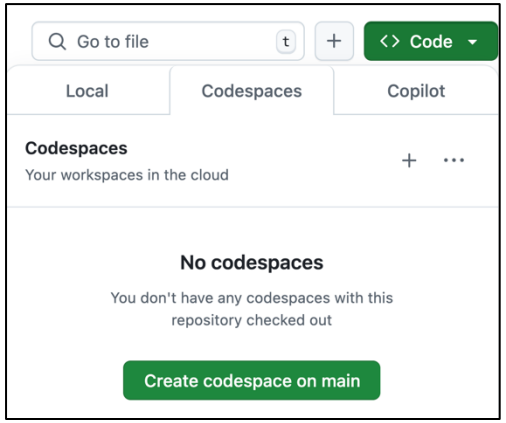
- What is context?
- Memory
  - Sessions
  - Chat history
  - Dynamic memory
- Knowledge providers
- Context management

Want to follow along?

1. Open this GitHub repository:

aka.ms/python-agentframework-demos

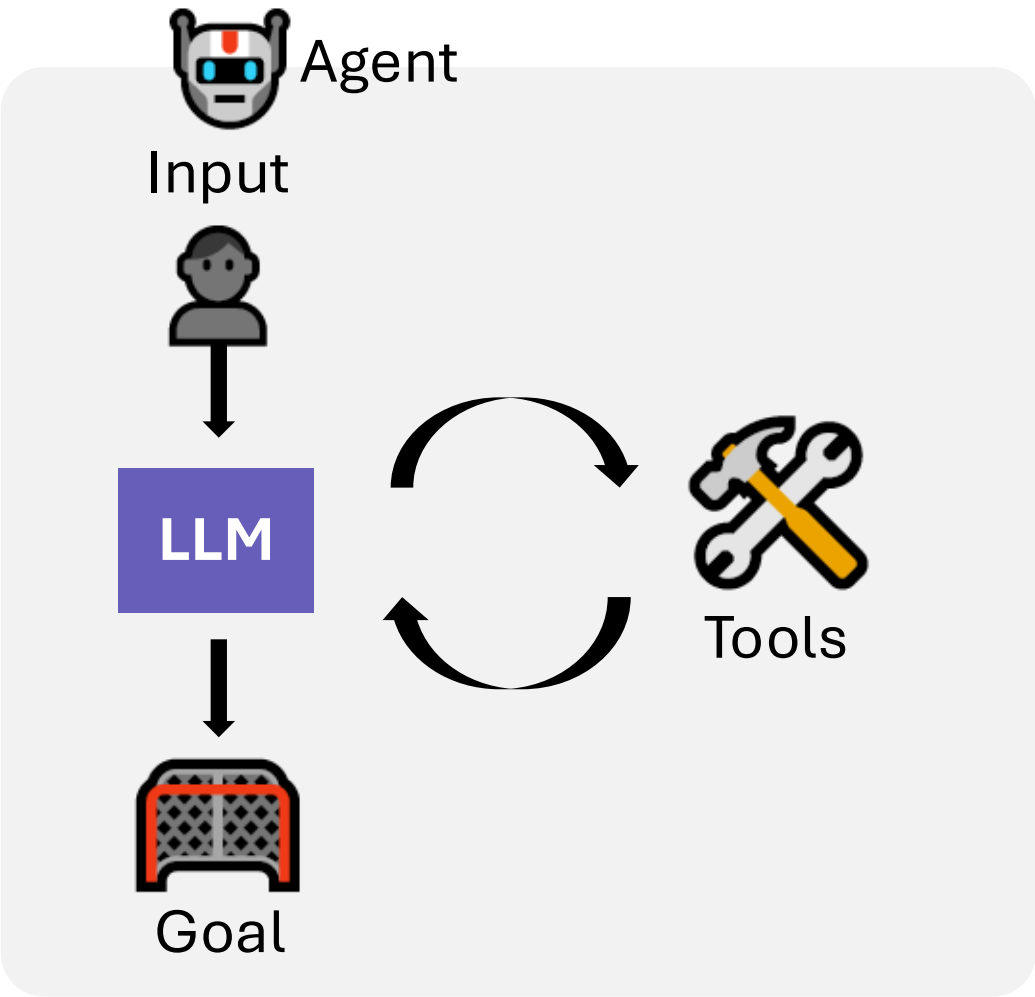
2. Use "Code" button to create a GitHub Codespace:



3. Wait a few minutes for Codespace to start up ⏱️
4. Pull the latest changes from the repo:

```
git pull
```

Recap: What's an agent?



An **AI agent** uses an **LLM** to run **tools** in a loop to achieve a **goal**.

Agents are improved by better **context**:

- Knowledge
- Memory
- Humans

Context is everything

The context to an agent is all the inputs that help the agent decide what action it will take next and how it will respond.

- Overall instructions
- Environment information
- Domain-specific knowledge
- Memories from previous conversations
- Past messages in the session
- Tool descriptions, calls, results

You're a friendly weather assistant.

Today's date is Feb 25, 2026.

There is an emergency alert today for CA citizens due to wildfire risk.

The current user lives in California.

Weather in Tokyo?

LLM

get_weather("Tokyo")

Sunny, 25°C

LLM

It's sunny in Tokyo at 25°C

🧠 Memory

Memory

Can LLM remember past messages?

Can app remember past conversations?

Can LLM remember facts from past conversations?

Session

- Within conversation, just messages and tool calls
- Can be stored in-memory OR persisted to a database to restore conversation later
- May require truncation or summarization

Chat history

- Must be stored in a persistent database
- Across conversations, typically tied to user ID

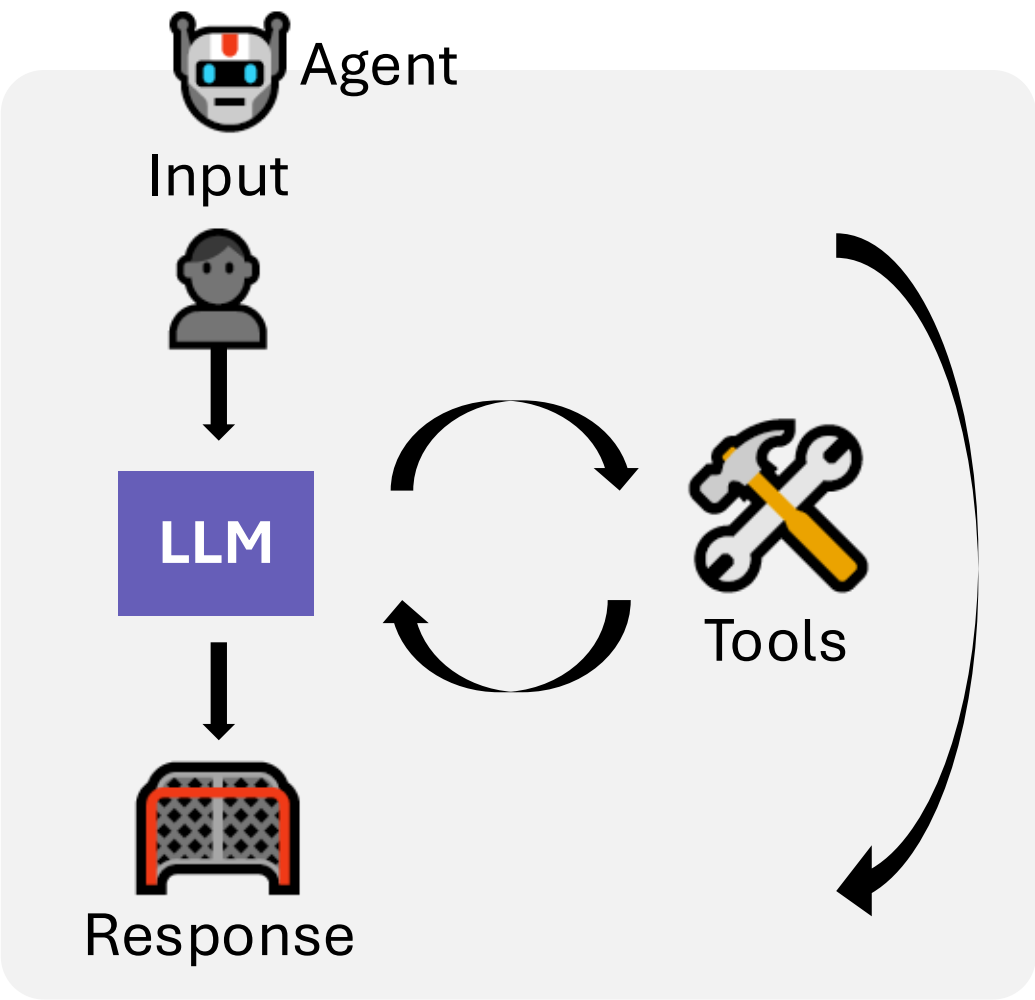
Dynamic memory

- Must be stored in a persistent database
- Retrieve memories, optionally using search to find most relevant

https://learn.microsoft.com/agent-framework/user-guide/agents/agent-memory

💬 Conversation sessions

Conversation sessions



A **session** includes all the past messages:

agent-framework

Conversation session (In-memory)

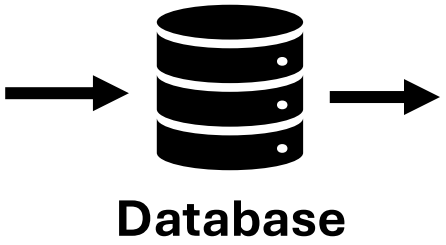
By default, a **session** stores the conversation messages in memory:

```
from agent_framework import Agent
from agent_framework.openai import OpenAIChatClient

agent = Agent(
  client=OpenAIChatClient(),
  instructions="You are a helpful assistant.")

session = agent.create_session()

response = await agent.run("Hello, my name is Alice", session=session)
```

[Full example: agent_session.py]

Persistent chat history

Persistent chat history

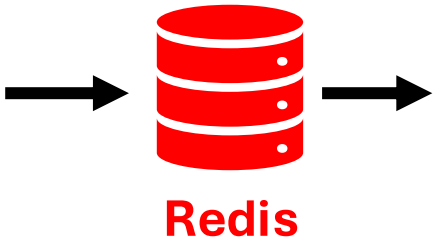


id	messages
session123	system: You're a weather assistant. user: Weather in Tokyo? ...
session456	system: You're a weather assistant. user: Will it rain in CA? ....

Implementation details:

- Will entire session fit in one row? If not, how to handle?
- How will sessions be linked to users?
- Will the sessions be searchable?

Persistent chat history in Redis



key	value [LIST]
session123	{"role": "system", "content": "You're a weather assistant."}
	{"role": "user", "content": "Weather in Tokyo?"}
	{"role": "assistant", "tool_call": "get_weather"}

Implementation details:

- Keys are session IDs
- The values are lists of messages that can be truncated to the most recent N

agent-framework

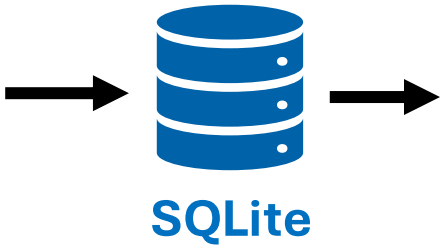
Persistent chat history

Redis support is built-in, so you can directly use `RedisHistoryProvider`:

```
from agent_framework.redis import RedisHistoryProvider

redis_provider = RedisHistoryProvider(
    redis_url="redis://localhost:6379",
    source_id="redis_chat")

agent = Agent(
  client=OpenAIChatClient(),
  instructions="You are a helpful weather agent.",
  tools=[get_weather],
  context_providers=[redis_provider])

session = agent.create_session(session_id=str(uuid.uuid4()))
```

[Full example: agent_history_redis.py]

Persistent chat history in SQLite



id	thread_id	message
1	session123	{"role": "system", "content": "You're a weather assistant."}
2	session123	{"role": "user", "content": "Weather in Tokyo?"}
3	session123	{"role": "assistant", "tool_call": "get_weather"}
4	session456	{"role": "system", "content": "You're a weather assistant."}

Implementation details:

- Rows are single messages, retrieved based off incrementing ID order
- No limits to # of messages in session

agent-framework

Persistent chat history with custom storage

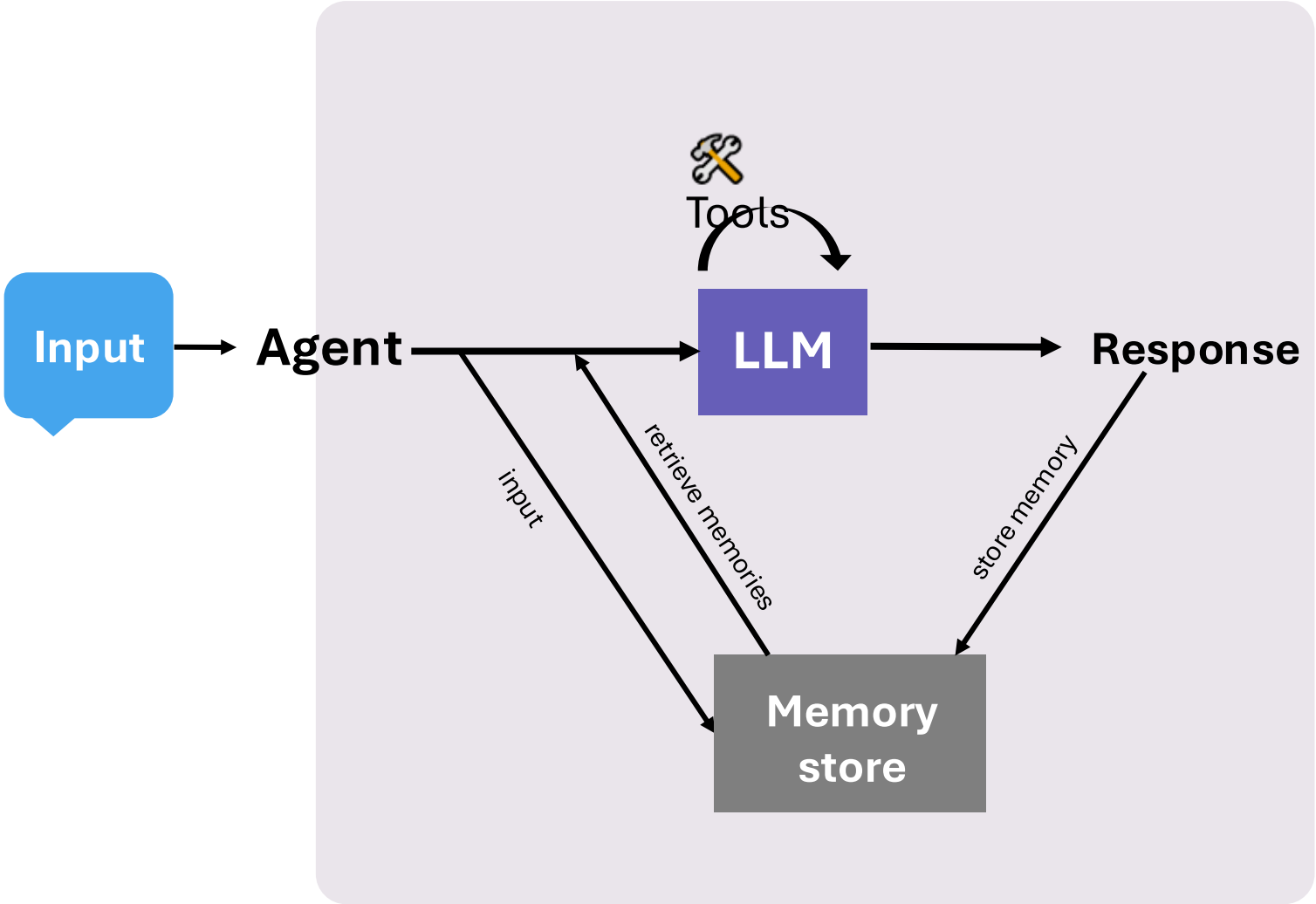
For DBs without built-in support, write a subclass that implements `BaseHistoryProvider`:

```
class SQLiteHistoryProvider(BaseHistoryProvider):

  def __init__(self, db_path: str):
    ...

  async def get_messages(self, session_id, **kwargs) -> list[Message]:
    cursor = self._conn.execute(
      "SELECT message_json FROM messages WHERE session_id = ? ORDER BY id",
      (session_id,))
   return [Message.from_json(row[0]) for row in cursor.fetchall()]

  async def save_messages(self, session_id, messages, **kwargs) -> None:
    self._conn.executemany(
      "INSERT INTO messages (session_id, message_json) VALUES (?, ?)",
      [(session_id, msg.to_json()) for msg in messages])
     self._conn.commit()
```

Full example: agent_history_sqlite.py

Dynamic memory

Dynamic memory



Implementation matters!

Retrieving memories:

- All or only most relevant?

Storing memories:

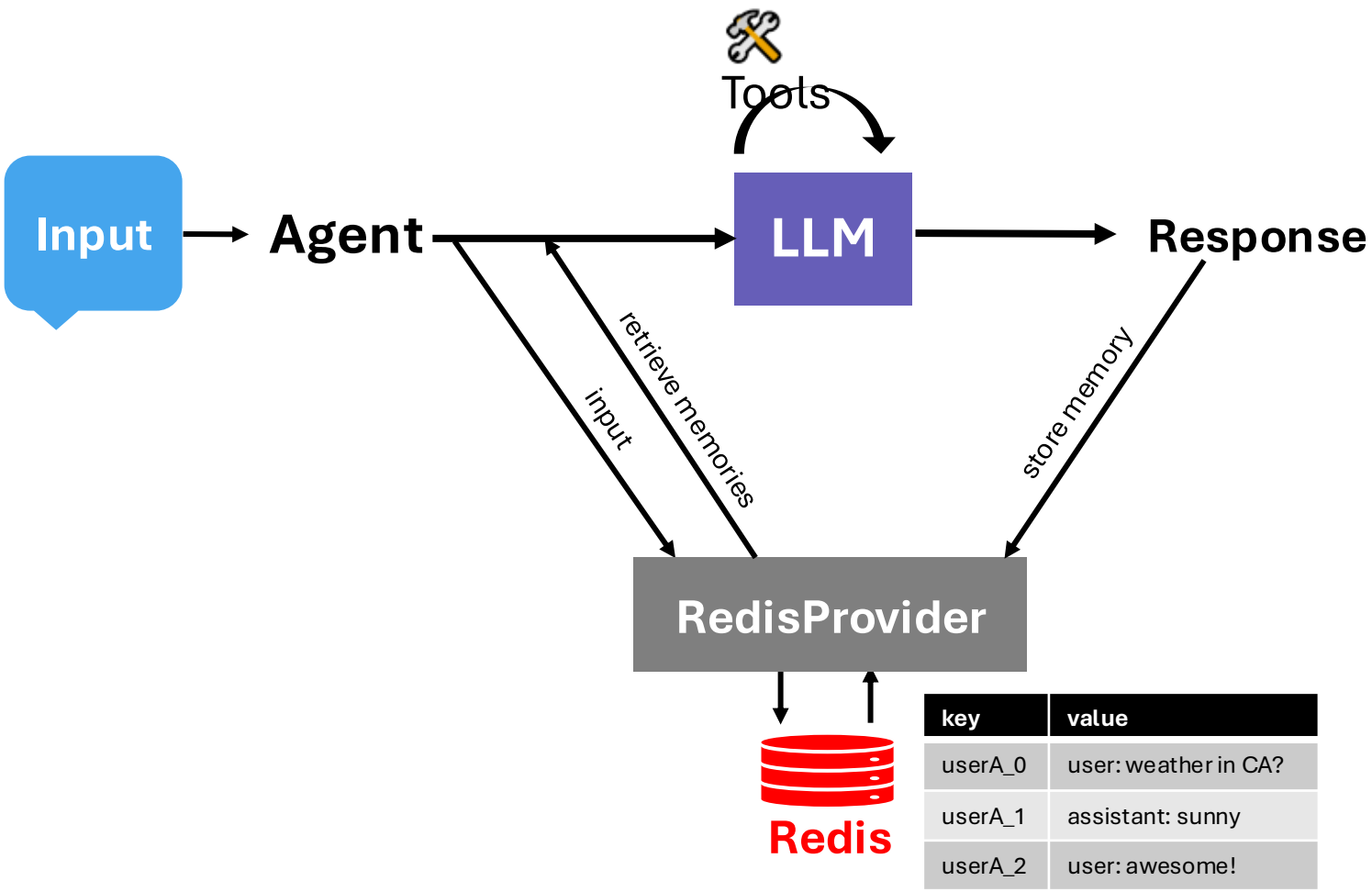
- Whole message?
- LLM-synthesized memory?

Forgetting memories:

- Oldest? Obsolete?

agent-framework

Dynamic memory with Redis



`RedisProvider:`

Retrieving memories:

- Searches with full-text, vector, or hybrid search

Storing memories:

- Entire user and assistant messages (no tool calls)

Forgetting memories:

- No mechanism- memories remain forever

agent-framework

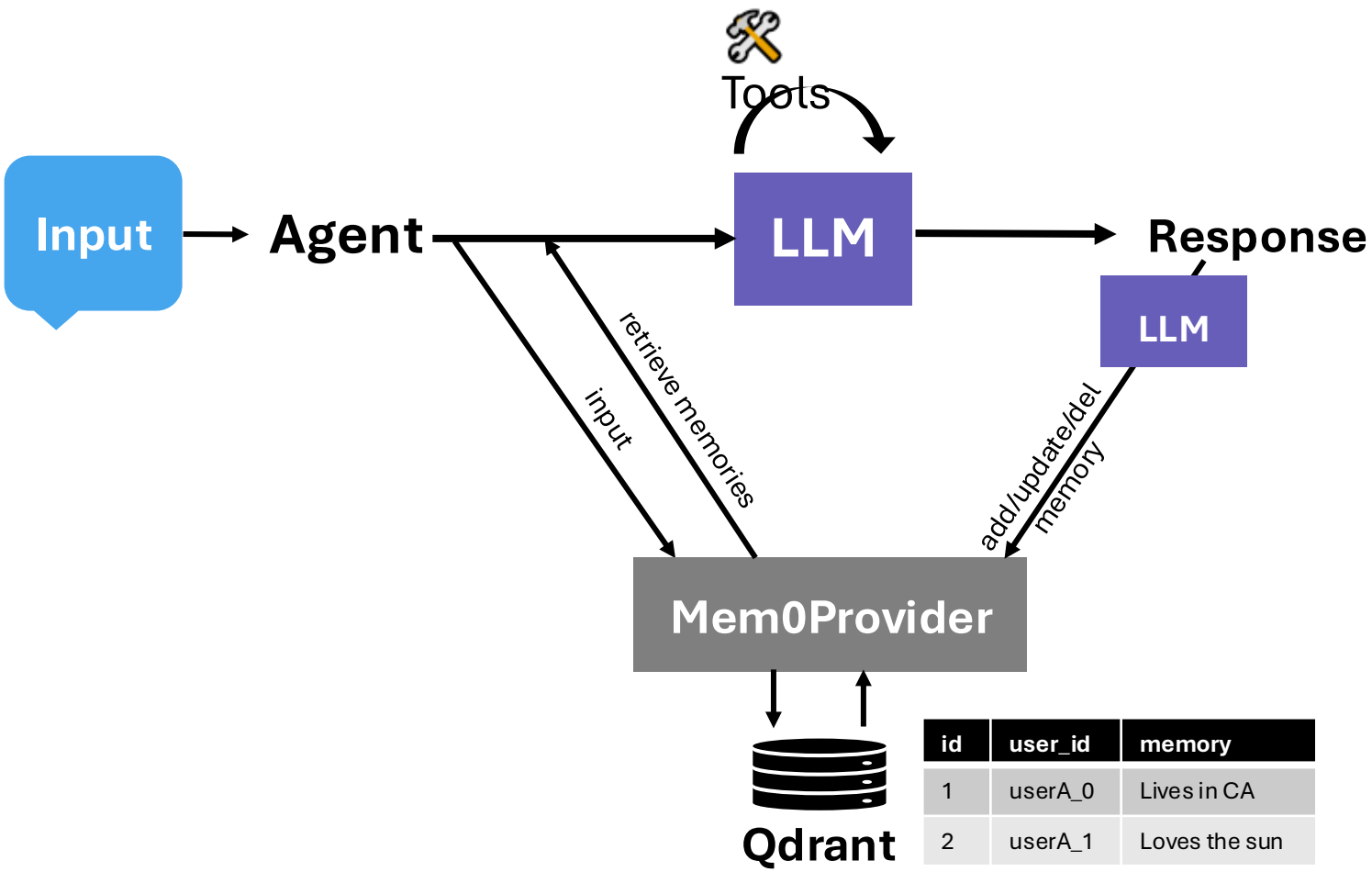
Dynamic memory with Redis

```
from agent_framework.redis import RedisContextProvider

user_id = str(uuid.uuid4())
memory_provider = RedisContextProvider(
  redis_url=REDIS_URL,
  index_name="agent_memory_demo",
  overwrite_index=True,
  prefix="memory_demo",
  application_id="weather_app",
  agent_id="weather_agent",
  user_id=user_id,
)
agent = Agent(
  client=OpenAIChatClient(),
  instructions="You are a helpful weather assistant.",
  context_providers=[memory_provider],
  tools=[get_weather])
```

Full example: agent_memory_redis.py

agent-framework

Dynamic memory with Mem0



id	user_id	memory
1	userA_0	Lives in CA
2	userA_1	Loves the sun

`Mem0Provider:`

Retrieving memories:

- Searches the associated database (Qdrant default)

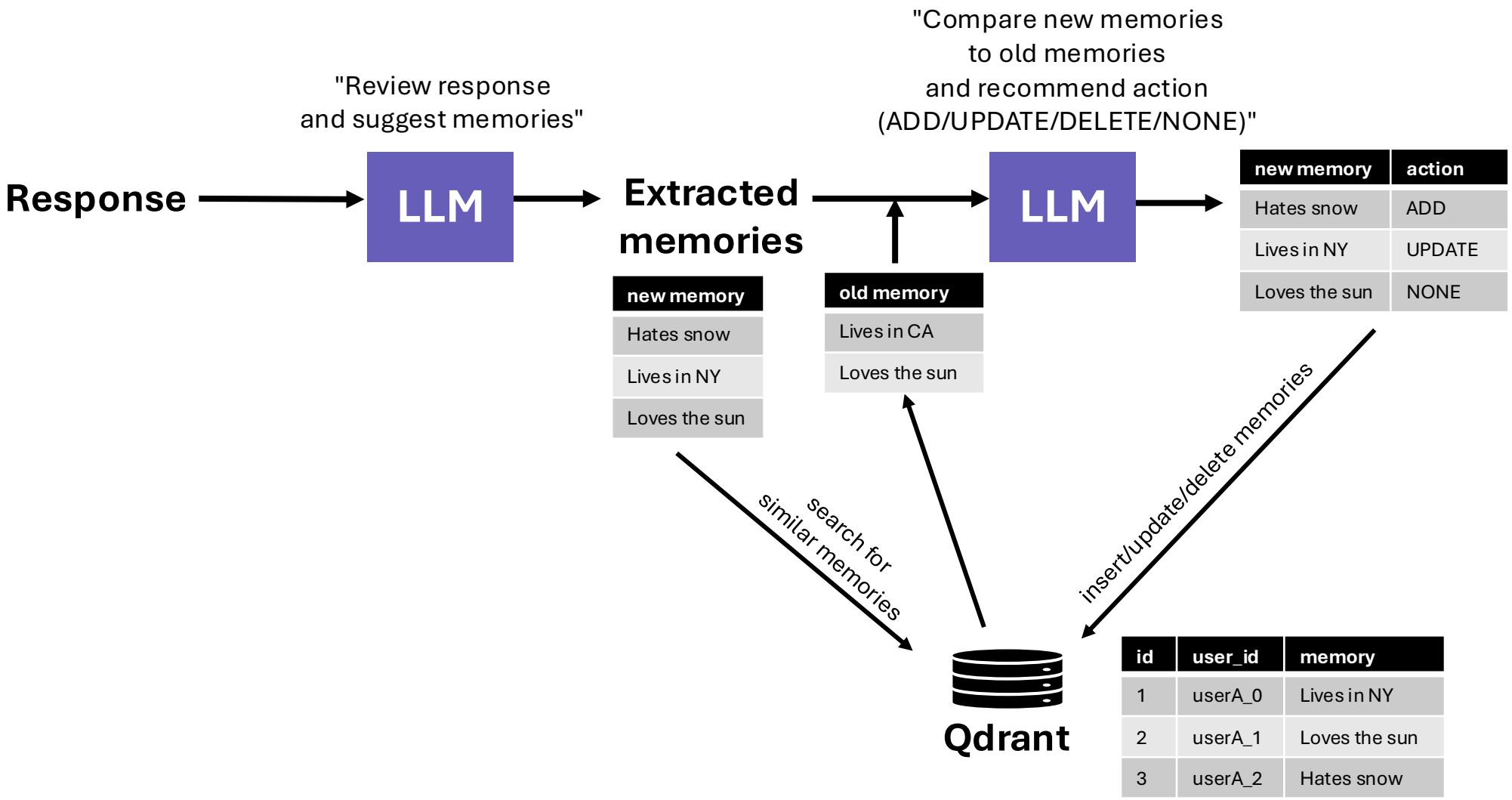
Storing memories:

- LLM suggests new memories

Forgetting memories:

- LLM suggests memory removal or updating

Mem0Provider: Memory update process



agent-framework

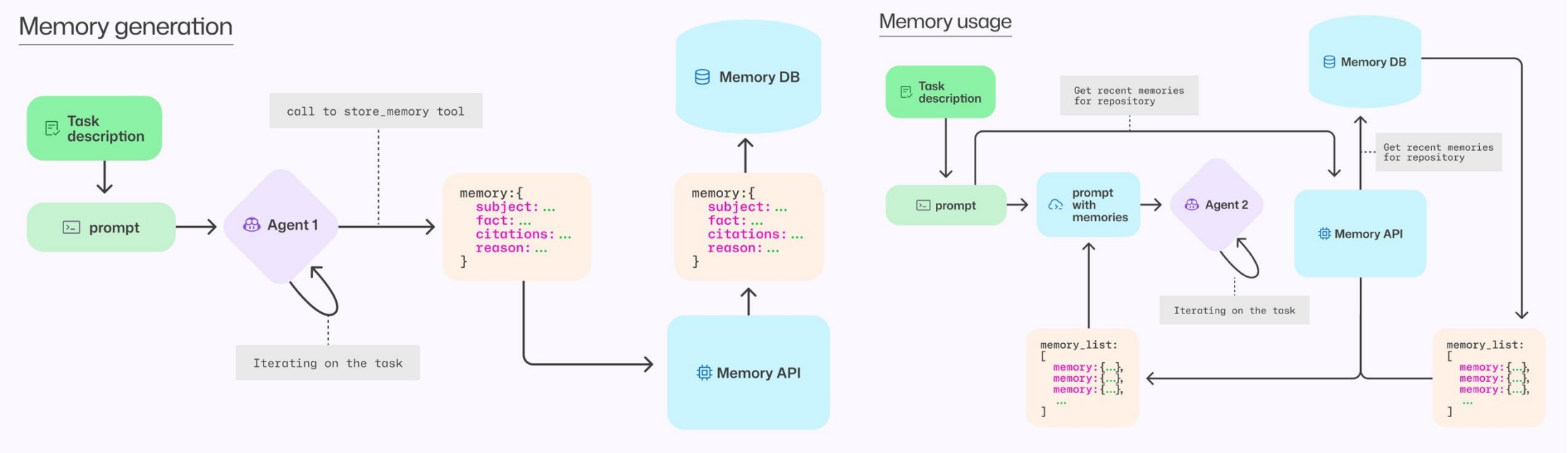
Dynamic memory with Mem0

Use a built-in memory provider (like `Mem0Provider`) or write your own context provider:

```
from agent_framework.mem0 import Mem0ContextProvider
from mem0 import AsyncMemory

user_id = str(uuid.uuid4())

mem0_client = await AsyncMemory.from_config(mem0_config)

memory_provider = Mem0ContextProvider(user_id=user_id, mem0_client=mem0_client)

agent = Agent(
  client=OpenAIChatClient(),
  instructions="You are a helpful weather assistant.",
  context_providers=[memory_provider],
  tools=[get_weather])
```

Full example: agent_memory_mem0.py

Memory in production: GitHub Copilot

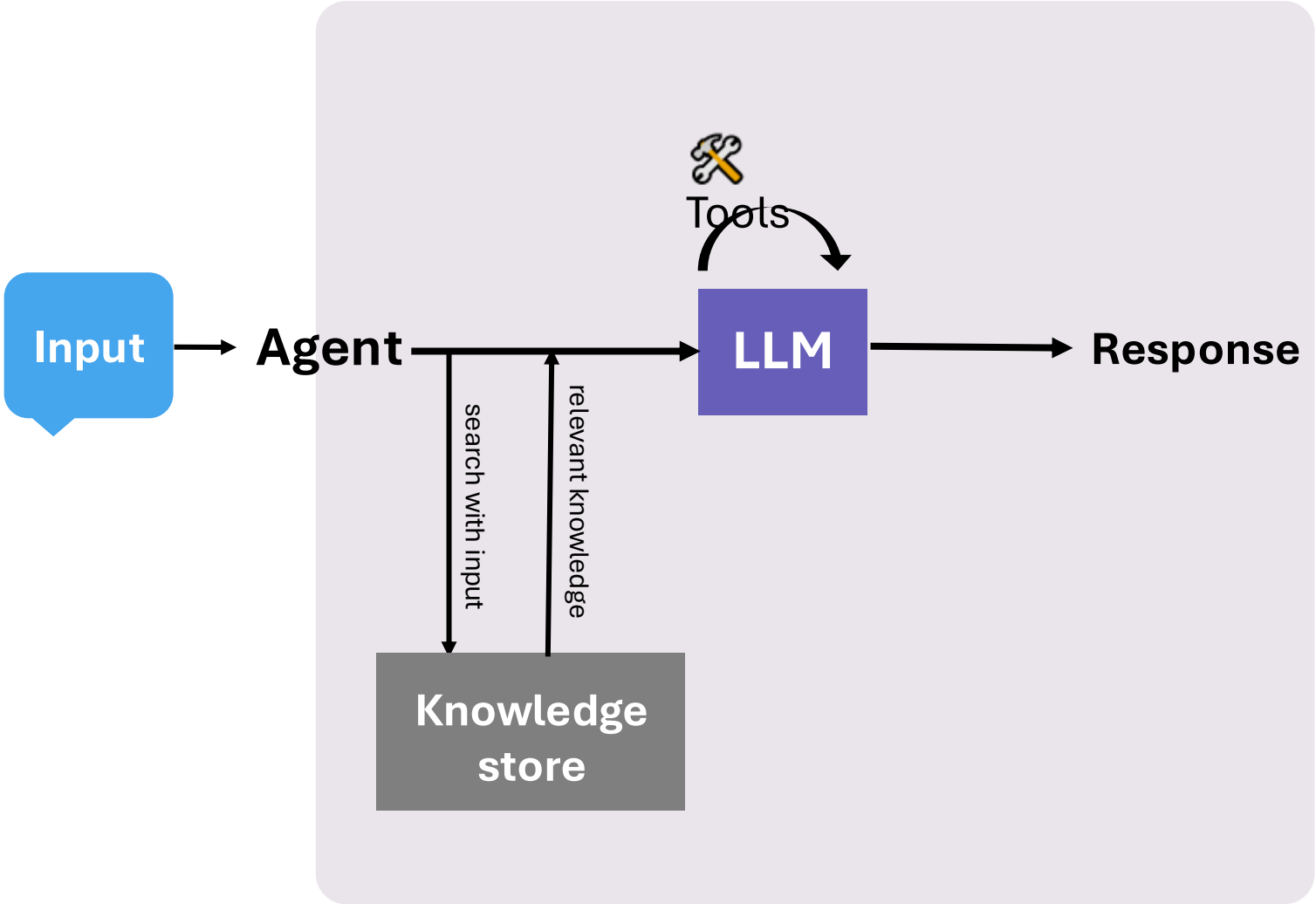


All memories include **citations**, which are verified for accuracy *before* memories are used. A memory which can no longer be verified is removed.

Learn more:
https://github.blog/ai-and-ml/github-copilot/building-an-agentic-memory-system-for-github-copilot/

Knowledge

Knowledge retrieval



The agent retrieves knowledge *before* asking the LLM to call tools.

Why not just use a tool to retrieve knowledge?

In many scenarios, the agent *always* needs domain-specific knowledge to ground its response, so it's unnecessary to ask an LLM to decide to use the tool.

agent-framework

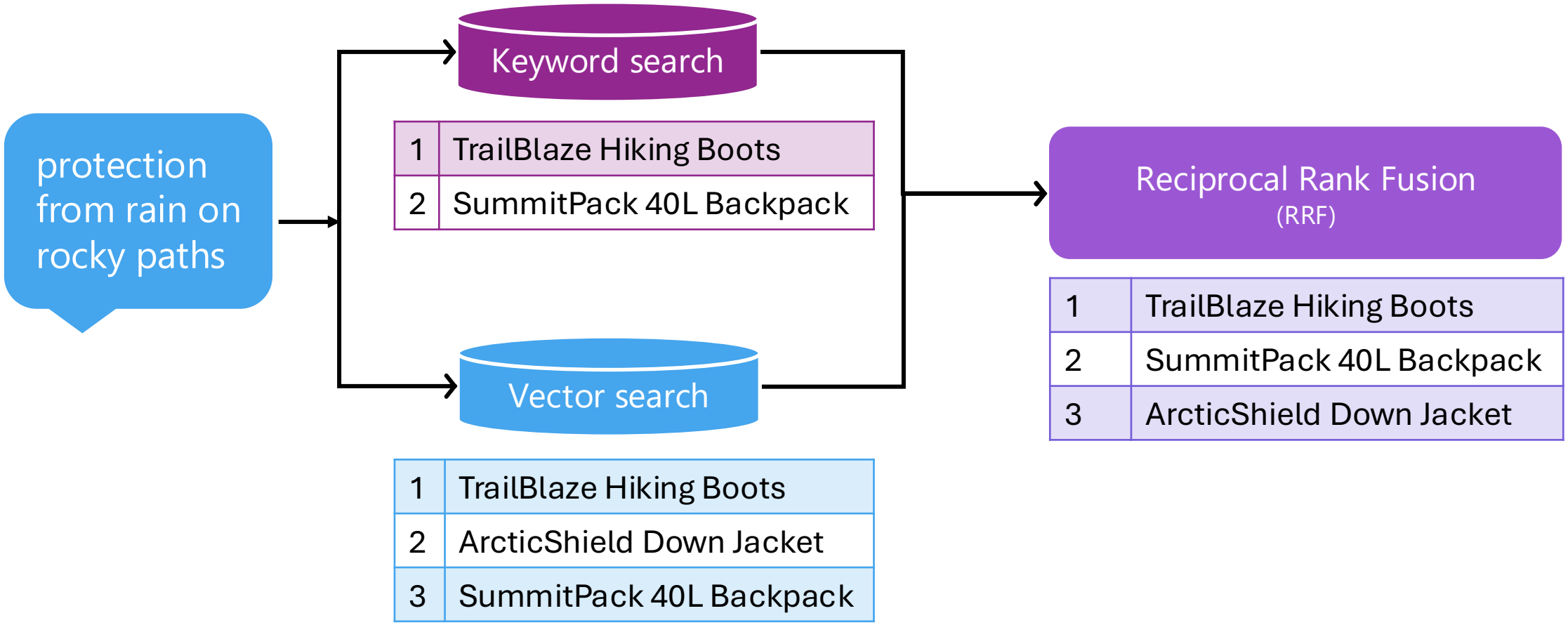
Knowledge retrieval from SQLite database

Create a subclass of `BaseContextProvider` to retrieve data for the given input:

```
class SQLiteKnowledgeProvider(BaseContextProvider):
  async def before_run(self, *, agent, session, context, state):
    user_text = context.input_messages[-1].text
    results = self._search(user_text)
    if results:
      context.extend_messages(self.source_id,
        [Message(role="user", text=self._format_results(results))])

agent = Agent(client=client,
  instructions=("You are an outdoor-gear shopping assistant."),
  context_providers=[knowledge_provider])

response = await agent.run("Do you have anything for kayaking?")
```

Full example: agent_knowledge_sqlite.py

Improve retrieval with hybrid search



agent-framework

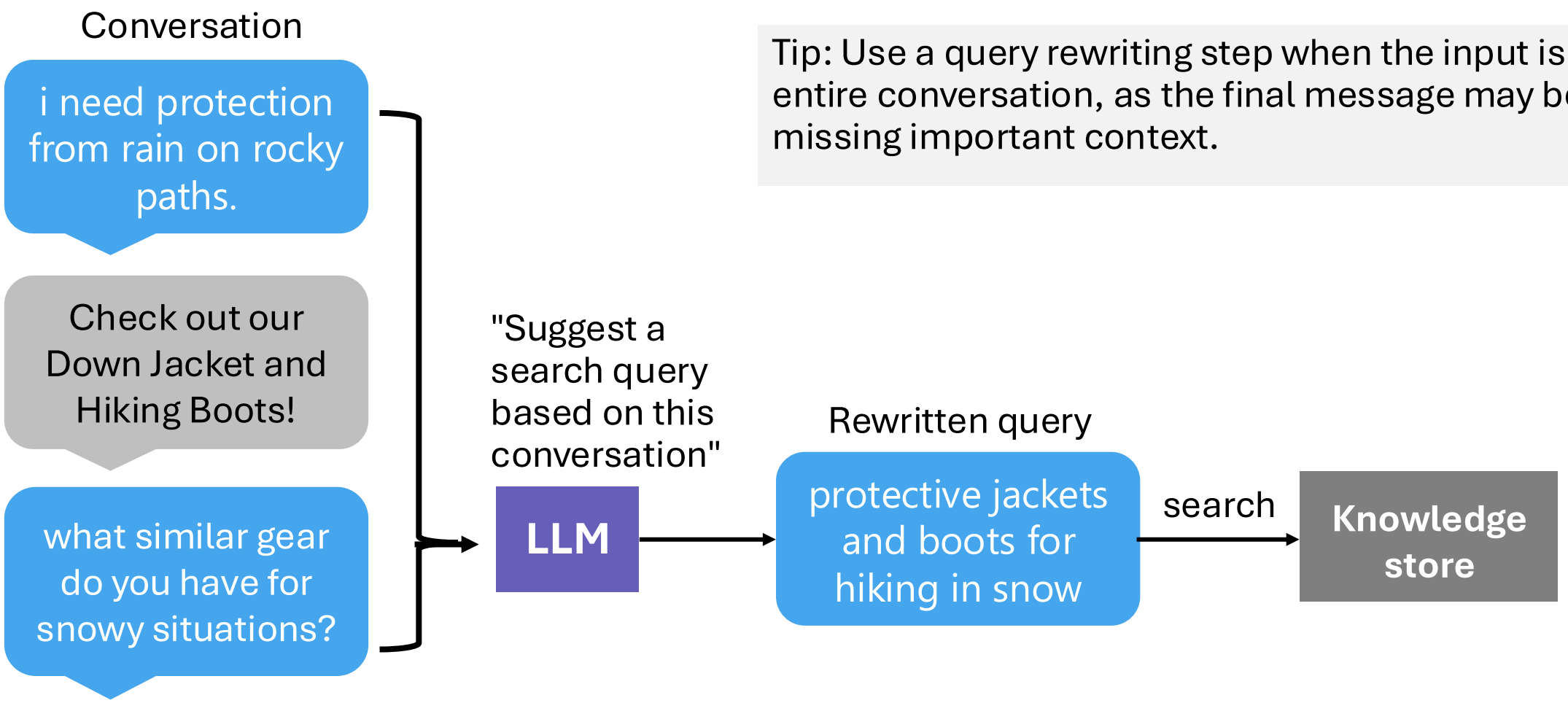
Knowledge retrieval from PostgreSQL

PostgreSQL can do a hybrid search with an embedding model and SQL query:

```
WITH semantic_search AS (
    SELECT id, RANK() OVER (ORDER BY embedding <=> %(embedding)s::vector(256)) AS rank
    FROM products
    ORDER BY embedding <=> %(embedding)s::vector(256)
    LIMIT 20
),
keyword_search AS (
    SELECT id, RANK() OVER (ORDER BY ts_rank_cd(
        to_tsvector('english', name || ' ' || description), query) DESC)
    FROM products, plainto_tsquery('english', %(query)s) query
    WHERE to_tsvector('english', name || ' ' || description) @@ query
    ORDER BY ts_rank_cd(...) DESC
    LIMIT 20
)
SELECT
    COALESCE(semantic_search.id, keyword_search.id) AS id,
    COALESCE(1.0 / (60 + semantic_search.rank), 0.0) +
    COALESCE(1.0 / (60 + keyword_search.rank), 0.0) AS score
FROM semantic_search
FULL OUTER JOIN keyword_search ON semantic_search.id = keyword_search.id
ORDER BY score DESC
LIMIT 3
```

Full example: agent_knowledge_postgres.py

Improve retrieval for multi-turn conversations



Tip: Use a query rewriting step when the input is an entire conversation, as the final message may be missing important context.

agent-framework

Query rewriting in a context provider

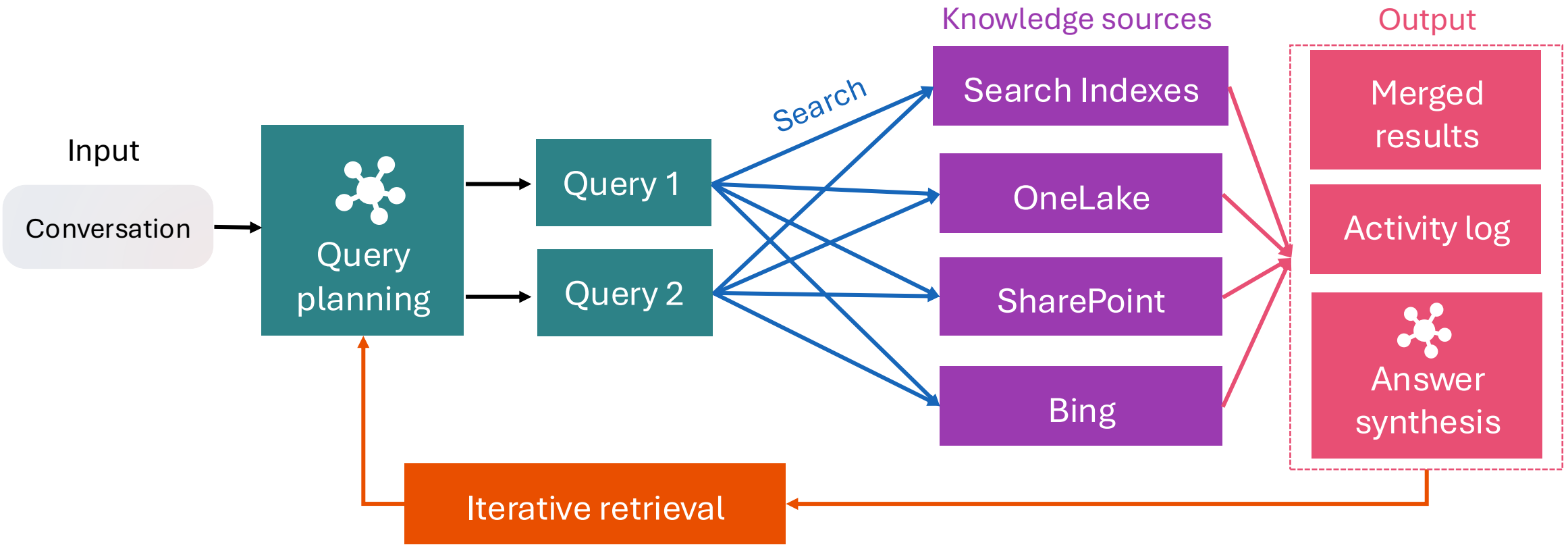
Use an LLM to rewrite the query based off the user/agent messages so far:

```
class PostgresQueryRewriteProvider(BaseContextProvider):

  async def before_run(self, agent, session, context, state):
    conversation = [
        m for m in context.get_messages() + context.input_messages
        if m.role in ("user", "assistant") and m.text]
   search_query = await self._rewrite_query(conversation)

   results = self._search(search_query)
   if results:
     context.extend_messages(self.source_id,
       [Message(role="user", text=self._format_results(results))])
```

[Full example: agent_knowledge_pg_rewrite.py]

Knowledge retrieval with Azure AI Search

Azure AI Search has a **knowledge base** feature which has **query rewriting** and **hybrid search** built-in, plus **multi-source selection** and a **reflection** step.



Search step uses hybrid search with semantic re-ranking when supported by the knowledge source

agent-framework

Knowledge retrieval from Azure AI Search

Use the built-in `AzureAISearchContextProvider` to retrieve from a knowledge base:

```
from agent_framework.azure import AzureAISearchContextProvider

search_provider = AzureAISearchContextProvider(
  endpoint=SEARCH_ENDPOINT, credential=DefaultAzureCredential(),
  knowledge_base_name=KNOWLEDGE_BASE_NAME, mode="agentic")

agent = Agent(
  client=client,
  instructions="You are a helpful shopping assistant.",
  context_providers=[search_provider])

async with search_provider:
  session = agent.create_session()
  response = await agent.run("What interior paint do you have?",
    session=session)
  response = await agent.run("What supplies do I need to apply it?",
    session=session)
```

Full example: agent_knowledge_aisearch.py

Context management

Context window

The **context window** of an LLM is the amount of input tokens that an LLM can process at a given time.

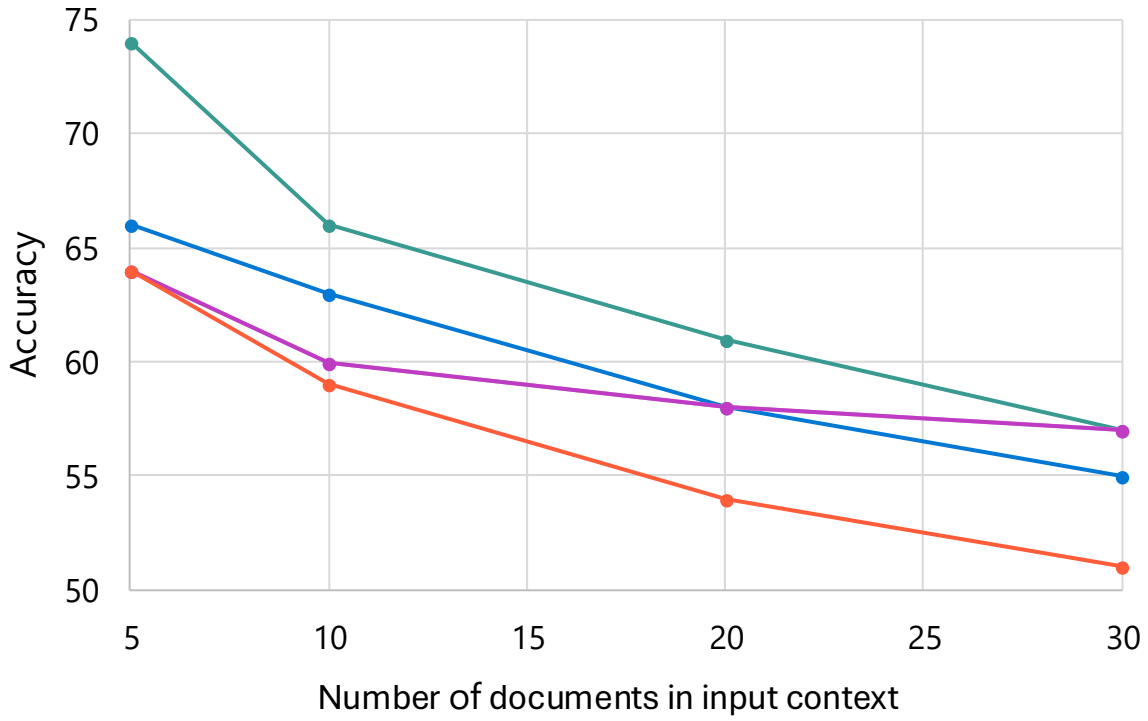
Context windows have been steadily increasing over time:

Model	Release date	Context window (tokens)
GPT-3	2020	2K
GPT-3.5 Turbo	Feb 2024	4K-16K
GPT-4	Mar 2023	8K-32K
GPT-4 Turbo	Dec 2023	128K
Claude 3 (Sonnet/Opus)	Feb 2025	200K
Claude Opus 4.6	Feb 2026	1M

Do we still need to worry about context size?

Absolutely!

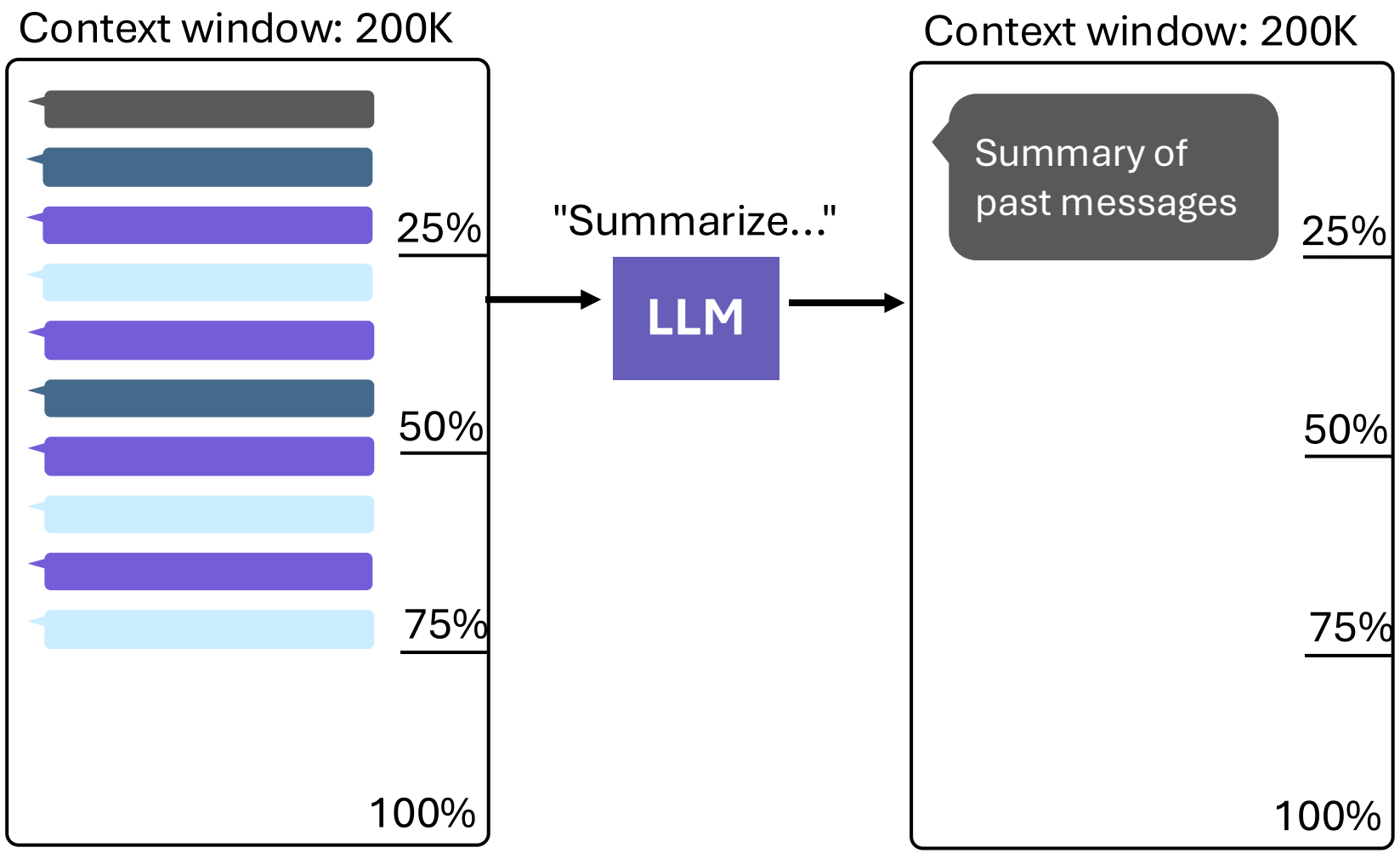
1. The context windows are finite (128K-2M)
2. When an LLM receives too much information, it can get easily distracted by irrelevant details.
3. The more tokens you send, the higher the cost, the slower the response.



Source: [Lost in the Middle]

See also: [Context rot research from Chroma]

Context compaction with summarization



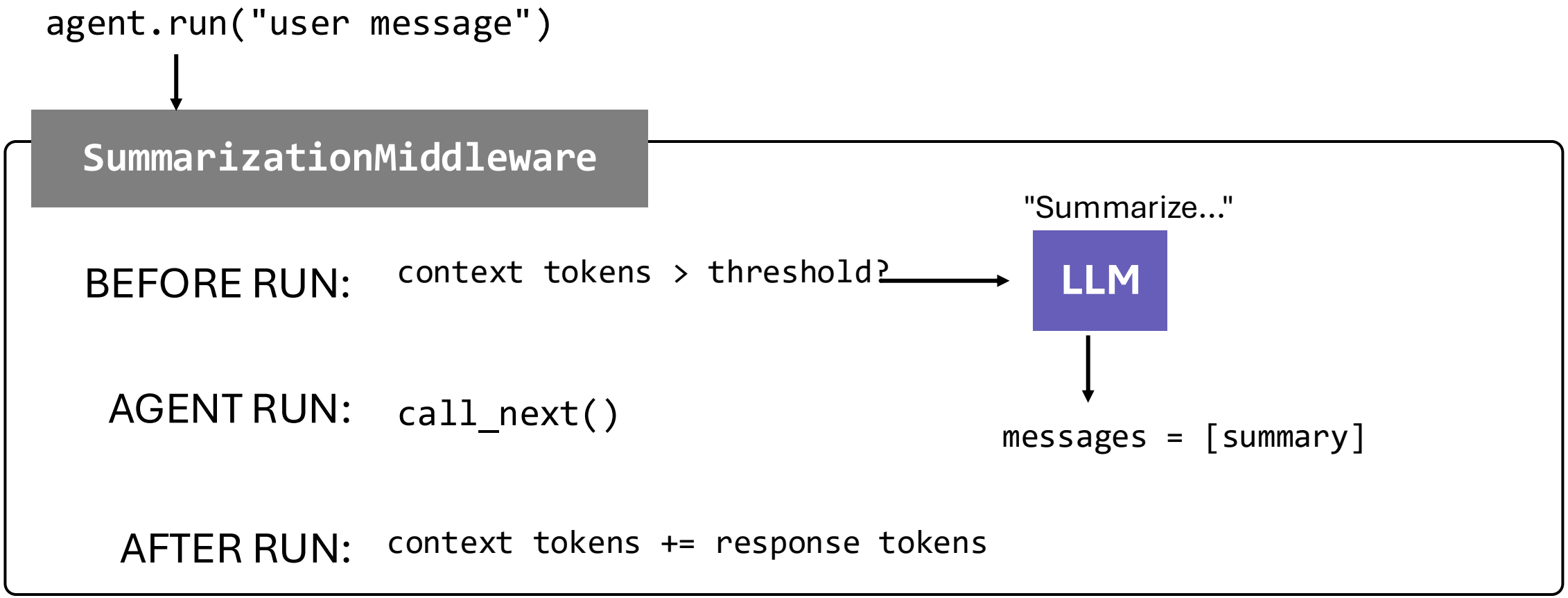
Implementation details:

- How often to summarize?
- What's most important to summarize?
- Do you store original messages for later?

agent-framework

Summarization middleware

We can subclass `AgentMiddleware` to summarize messages and replace them.



agent-framework

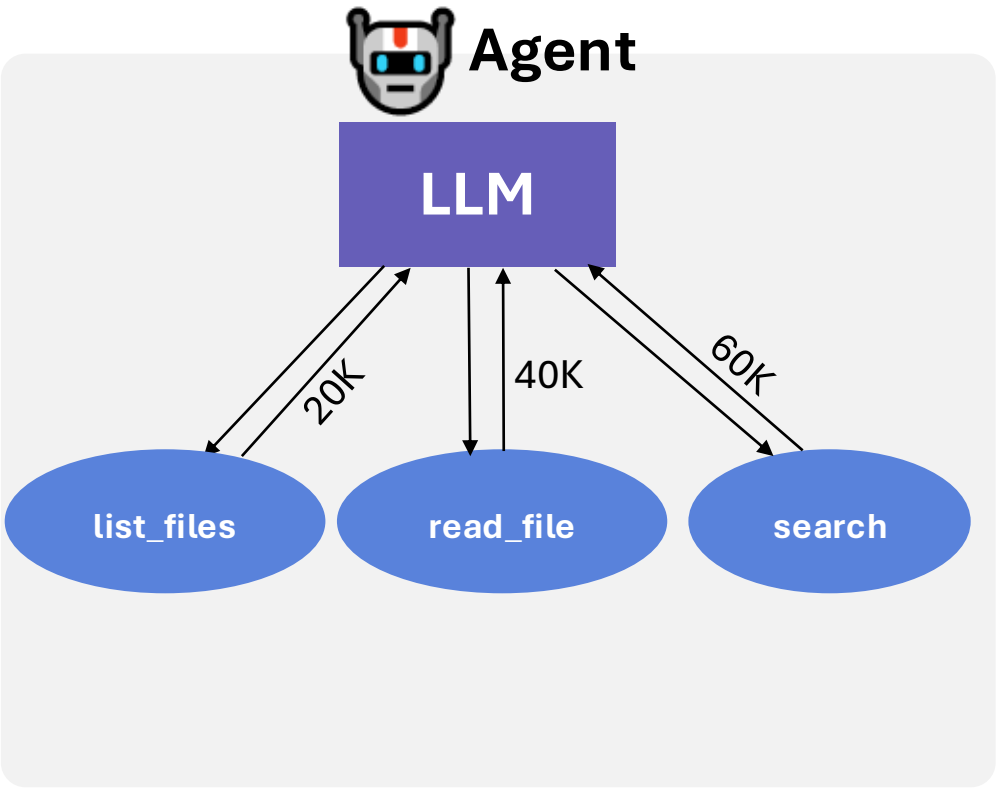
Summarization middleware

```
class SummarizationMiddleware(AgentMiddleware):

  async def process(self, context, call_next):
    # Before the agent runs
    if context.session and self.context_tokens > self.token_threshold:
      history = context.session.state["memory"]["messages"]
      summary = await self._summarize(history) # LLM call
      context.session.state["memory"]["messages"] =
       [Message(role="assistant", text=f"[Summary]\n{summary}")]
      self.context_tokens = 0

    await call_next() # Runs the standard agent loop

    # After the agent runs
    if context.result and context.result.usage_details:
      self.context_tokens += context.result.usage_details["total_token_count"]
```

Full example: agent_summarization.py

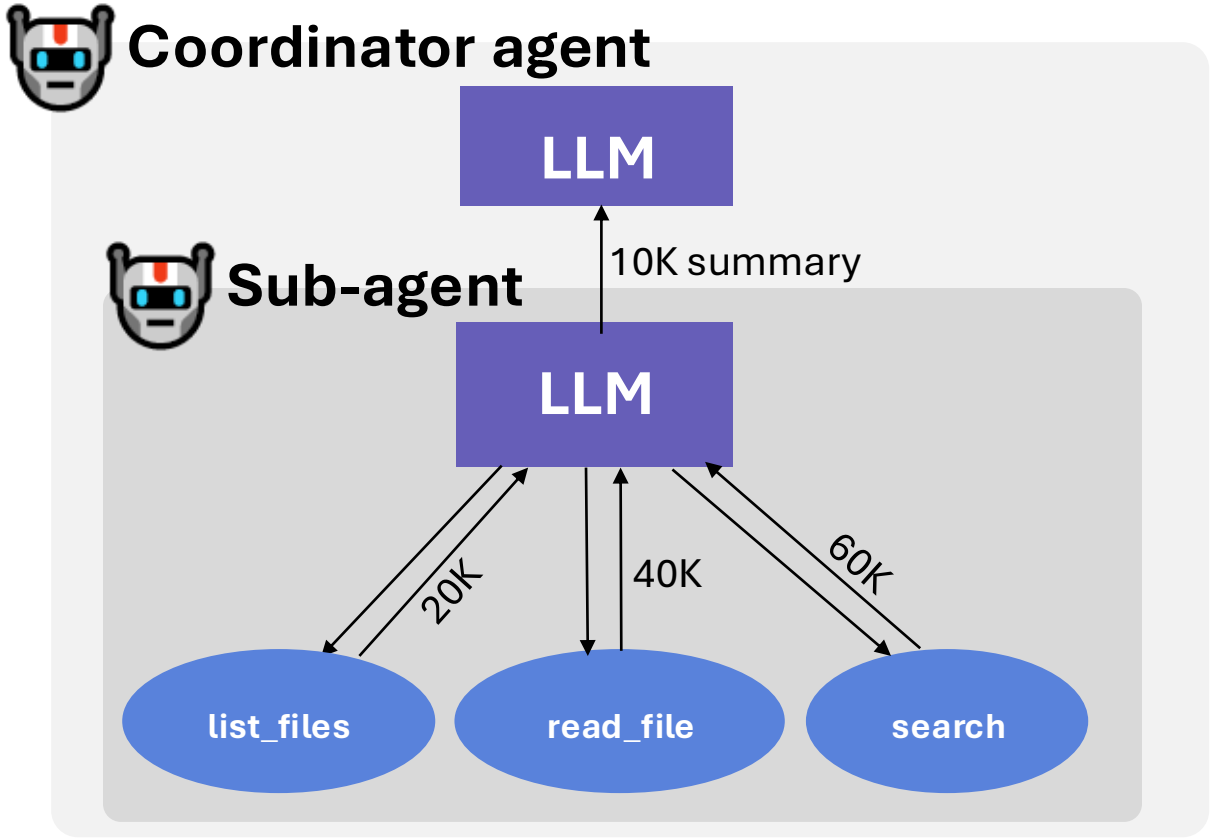
Context reduction with subagents

A single agent is risky...



😳 Context can explode due to large tool call results growing over time

Better: **Coordinator agent** calls **sub-agents**



Sub-agent returns a *summary* of tool call results.

agent-framework

Coordinator agent with sub-agents

```python
# Sub-agent: researches codebase in its own isolated context
research_agent = Agent(client=client,
  instructions="Read and search files, return a concise summary under 200 words.",
  tools=[list_project_files, read_project_file, search_project_files],
)

@tool
async def research_codebase(question: str) -> str:
  """Delegate research to the sub-agent. Returns only a summary."""
  response = await research_agent.run(question)
  return response.text

# Coordinator agent: only sees summaries, never raw file contents
coordinator = Agent(client=client,
  instructions="Answer questions about code. Use research_codebase to investigate.",
  tools=[research_codebase])
```

Full example: agent_with_subagent.py

Context reduction with subagents: Token usage

The coordinator's context window stays small, despite sub-agent using more tokens. In multi-turn conversations, this difference would compound with each turn.

	Single agent	vs.	Coordinator agent	Sub-agent
Input tokens	6,714		623	8,494
Output tokens	1,598		514	580
Total tokens	8,312		1,137	9,074

The final answer returned was a high-quality answer in *both* scenarios.

When to use sub-agents?

✅ **Use sub-agents when...**

- Tools return large outputs
- Task requires multiple chained tool calls
- Main agent has multiple responsibilities
- Long-running, multi-turn conversations
- Sub-tasks need specialized instructions

❌ **Skip sub-agents when...**

- Tool responses are small and simple
- Single tool call gets the answer
- Main agent needs to see raw tool output
- Low latency is critical (extra LLM round-trip)

Next steps

Register:
https://aka.ms/PythonAgents/series

Watch past recordings:
aka.ms/pythonagents/resources

Join office hours after each session in Discord:
aka.ms/pythonai/oh

🛠️ Feb 24: Building your first agent in Python
🧠 Feb 25: Adding context and memory to agents
📊 Feb 26: Monitoring and evaluating agents
🔀 Mar 3: Building your first AI-driven workflows
🕸️ Mar 4: Orchestrating advanced multi-agent workflows
🙋🏽‍♀️ Mar 5: Adding a human-in-the-loop to workflows